AFEKA TRIPS 2026 – WEB PROJECT

GitHub: https://github.com/nikolpinchevsky/WEB-final-project

Addresses in the cloud:

Frontend (Next.js): http://localhost:3000
Backend (Express): http://localhost:4000

מגישות: ניקול פינצבסקי + מאי שבת

בעיות ידועות / KNOWN BUGS

- במקרים נדירים יצירת מסלול ע"י ה-LLM יכולה להחזיר מסלול לא אופטימלי.
- זמן טעינת תחזית מזג האוויר תלוי בזמינות ה-API החיצוני.
- אם ה-refresh token פג תוקף בזמן עבודה ממושכת המשתמש יועבר מחדש ל-login.
- המערכת מותאמת בעיקר לדפדפני Chrome / Edge.

SYSTEM ARCHITECTURE

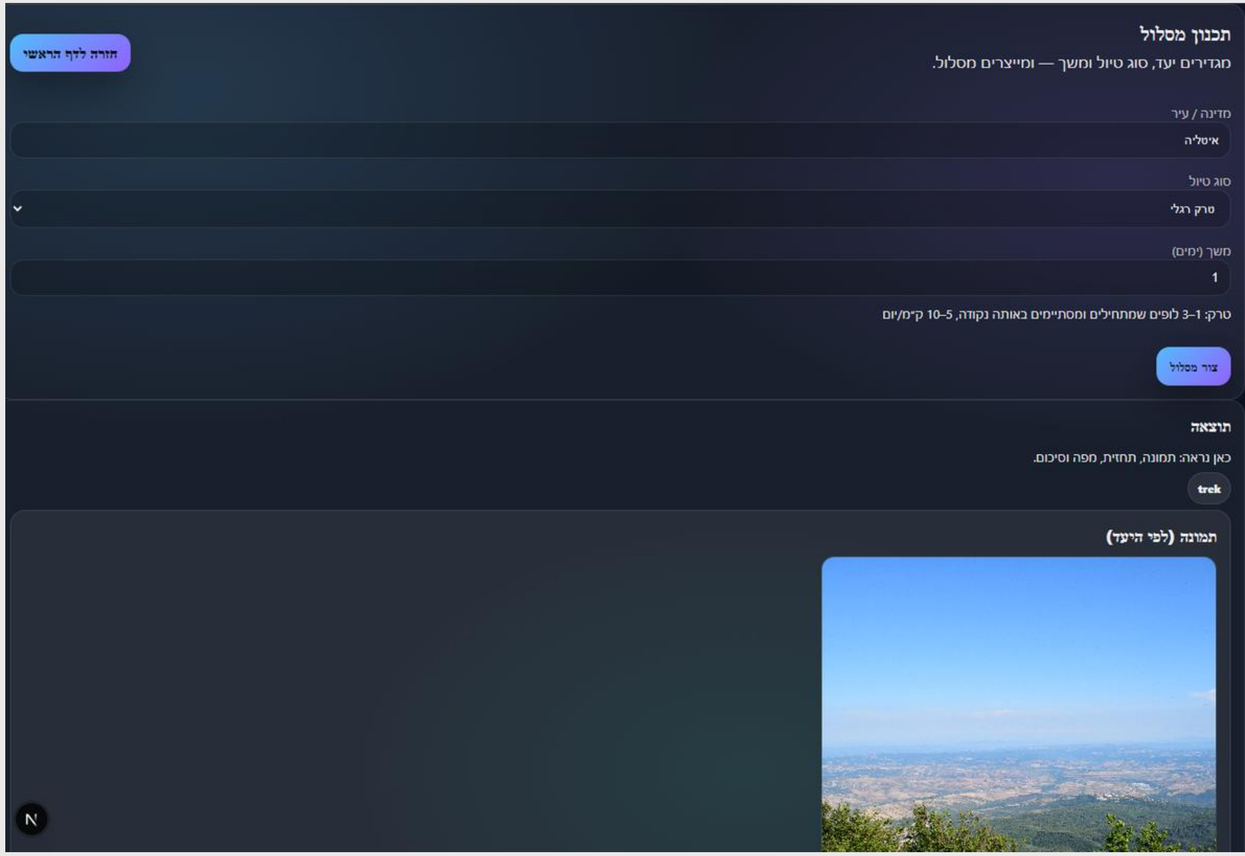
External APIs:

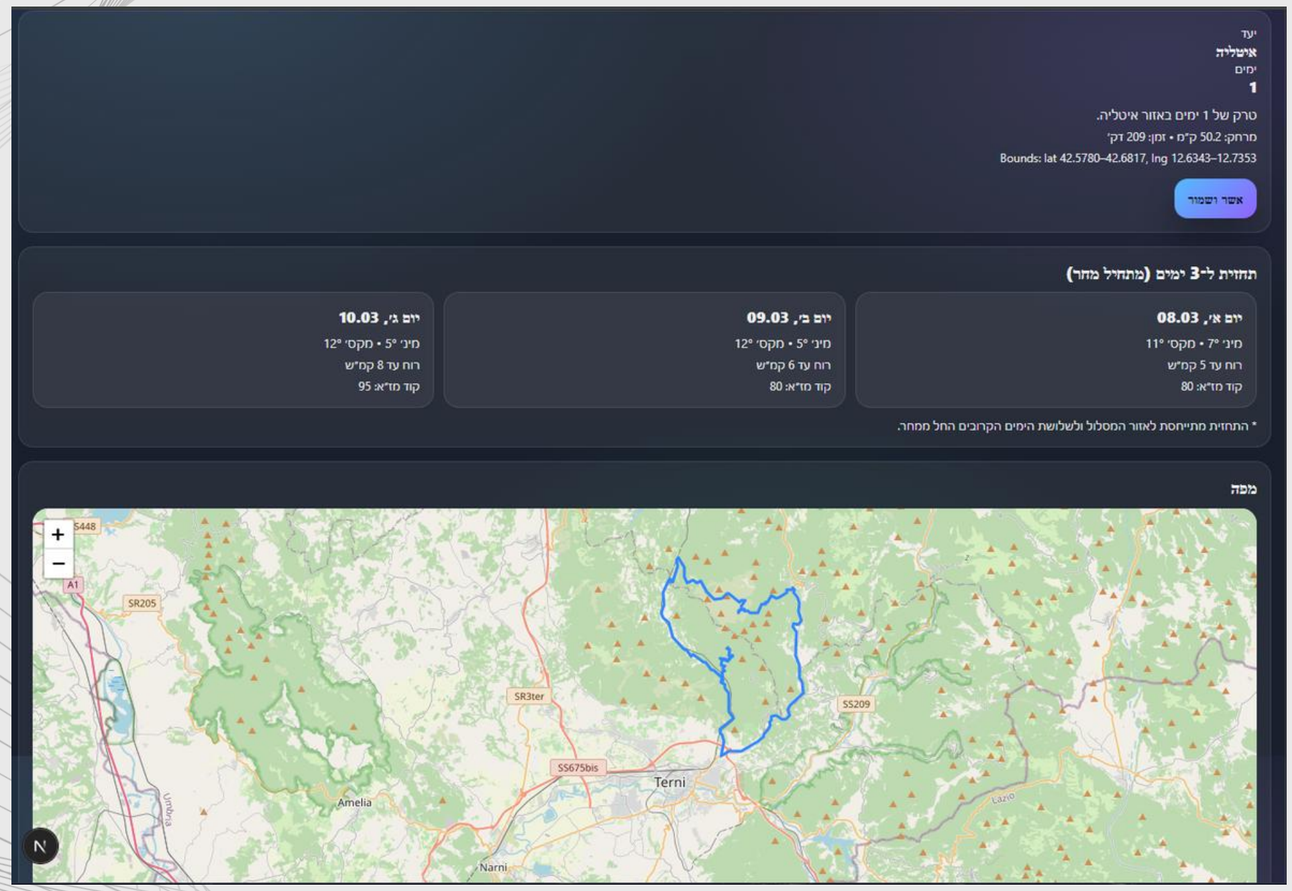
LLM API ➔ יצירת מסלולים

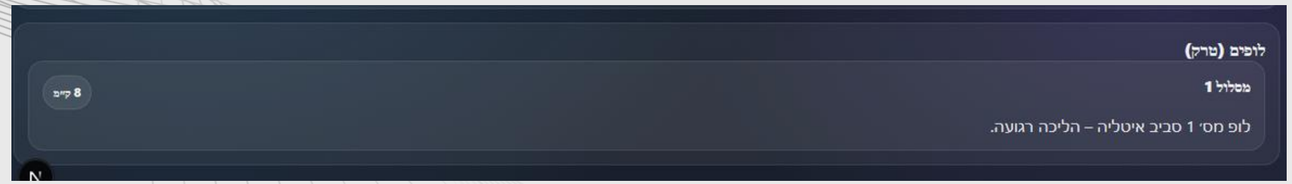
Weather API ➔ תחזית מזג אוויר

Image API ➔ תמונת המדינה

PLANNER PAGE FEATURES



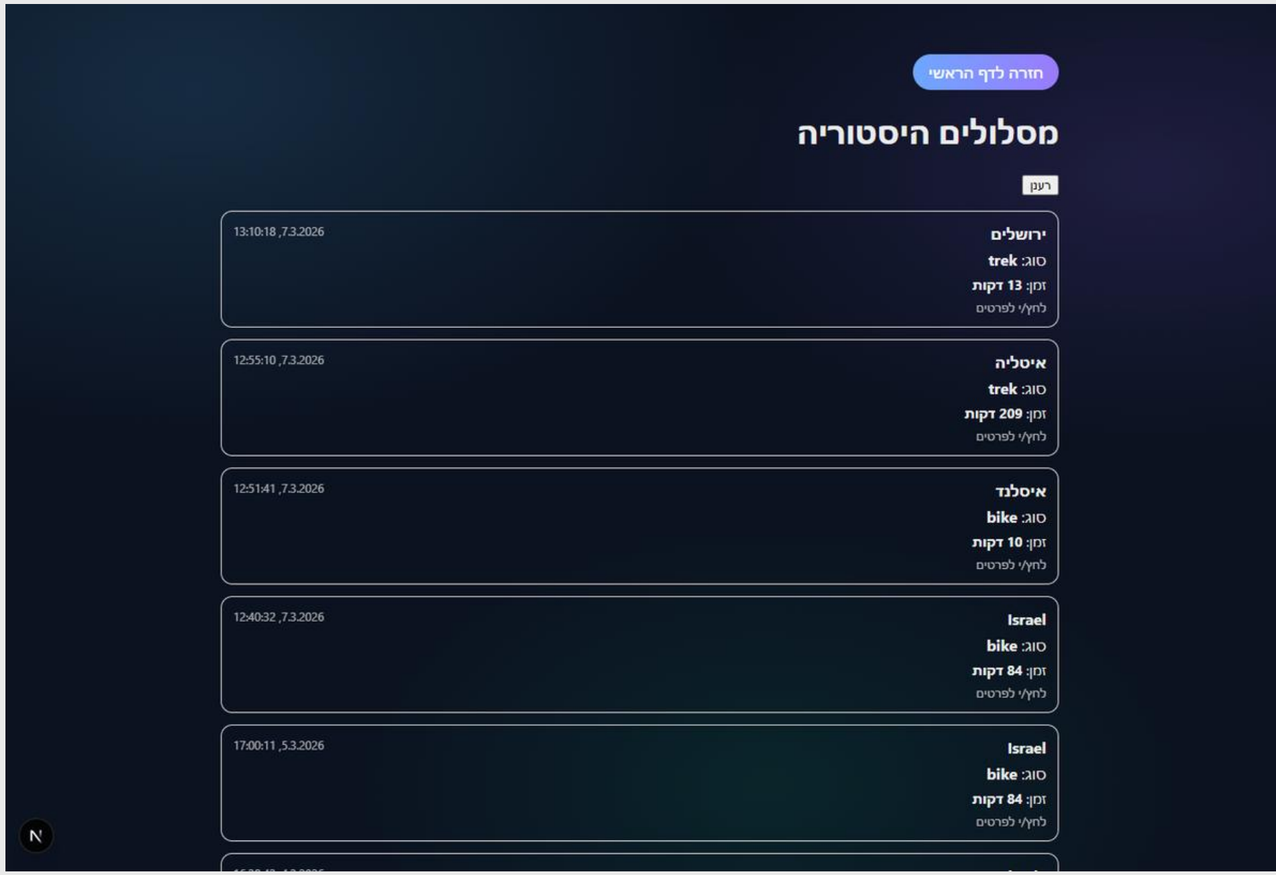


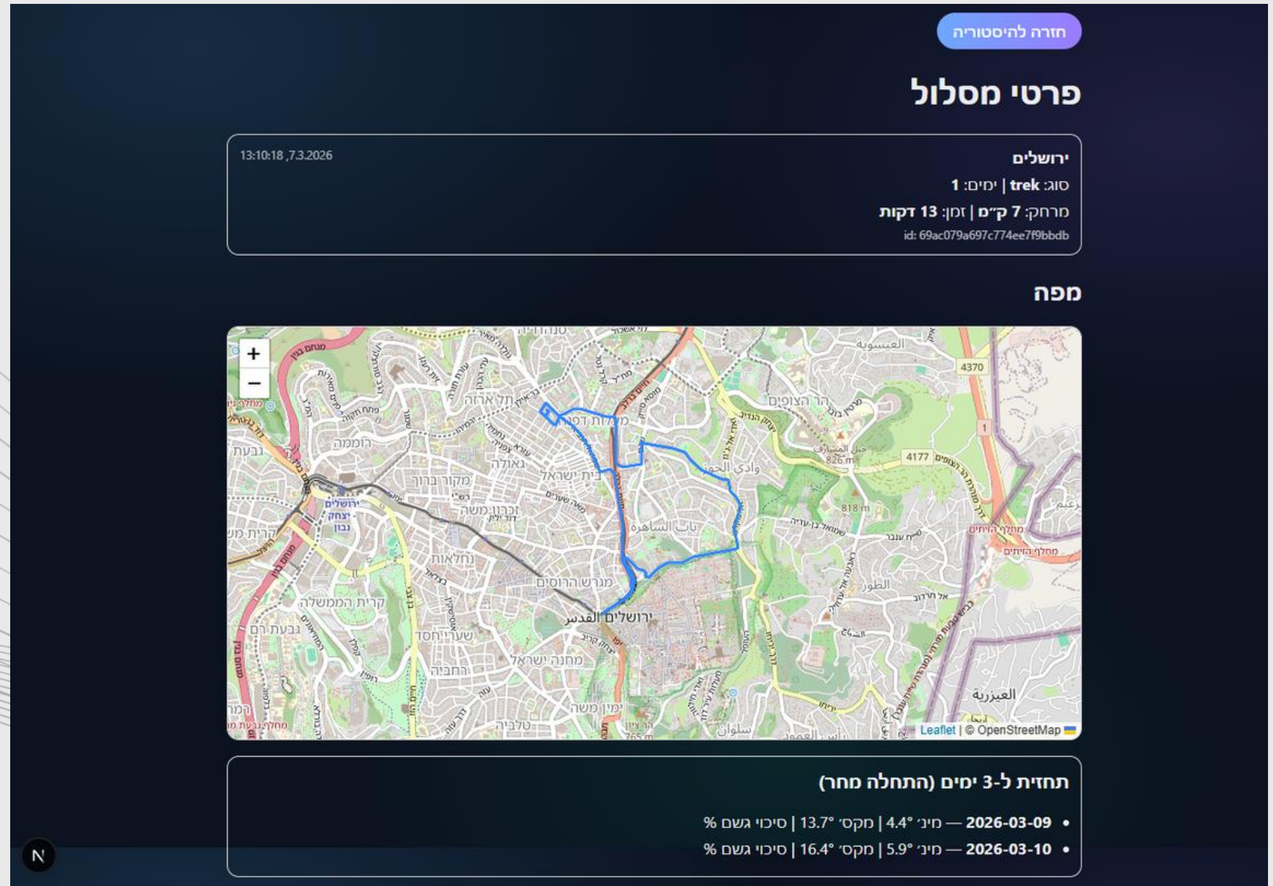


- יצירת מסלולי טיול בעזרת LLM
- הצגת המסלול על מפה (Leaflet)
- הצגת תחזית מזג אוויר
- אפשרות שמירה למסד נתונים

HISTORY PAGE

- שליפת מסלולים שנשמרו בעבר
- הצגת המסלול על המפה
- הצגת תחזית מזג אוויר עדכנית





IMPORTANT CODE SNIPPETS
AUTHENTICATION & SECURITY

JWT TOKEN GENERATION AFTER LOGIN:

```
const payload = { email: user.email, name: user.name };
const accessToken = signAccessToken(payload);
const refreshToken = signRefreshToken(payload);

setAuthCookies(res, { accessToken, refreshToken });
return res.json({ ok: true, user: payload });
```

MIDDLEWARE AUTHENTICATION CHECK:

```
const isProtected = PROTECTED.some((p) => pathname.startsWith(p));
if (!isProtected) return NextResponse.next();

const access = req.cookies.get("access_token")?.value;

if (!access) {
  const refreshed = await tryRefresh(req);
  if (refreshed) return refreshed;
  return NextResponse.redirect(new URL("/login", req.url));
}

return NextResponse.next();
```

IMPORTANT CODE SNIPPETS

CORE PROJECT LOGIC

REAL ROUTE GENERATION USING OSRM:

```
const profile = osrmProfile(tripType);
const coords = points.map((p) => `${p.lng},${p.lat}`).join(";");
const url = `https://router.project-osrm.org/route/v1/${profile}/${coords}?overview=full&geometries=geojson`;

const r = await fetch(url);
const data = await r.json();
```

LOADING TRIP HISTORY WITH UPDATED FORECAST:

```
const list = await trips
  .find({ userId: req.user.email })
  .sort({ createdAt: -1 })
  .limit(50)
  .toArray();

const enriched = [];
for (const t of list) {
  const trip = { ...t, _id: String(t._id) };
  try {
    const center = trip.center || (trip.geometry?.[0] ? trip.geometry[0] : null);
    if (center?.lat && center?.lng) {
      trip.forecast = await getForecast3DaysFromTomorrow(center);
    } else {
      trip.forecast = [];
    }
  } catch {
    trip.forecast = [];
  }
  enriched.push(trip);
}
```

SAVING APPROVED TRIP TO DATABASE:

```
const doc = {
  userId: req.user.email,
  ...toSave,
  createdAt: Date.now(),
};

const result = await trips.insertOne(doc);
return res.json({ ok: true, id: String(result.insertedId) });
```